PROJECT 4

Traffic Accident Severity Analytics and Prediction

Course: Data Engineering and MLOps | **Mode:** Individual Project | **Total Marks:** 100

Assignment Part 1 Submission Deadline: On or before 7 September 2026 (First Week of September)

1. Understanding the Project

Create a data pipeline that combines public accident records with weather and location-related information. Part 1 should produce an accident analytics warehouse and dashboard; Part 2 should operationalize a model that predicts accident severity.

Project objectives

- Integrate accident and weather datasets.
- Create a location- and time-aware analytical database.
- Visualize accident patterns and contributing factors.
- Build an MLOps workflow for severity classification.

2. Data Source Information

Data source	Information available	Suggested use
UK Road Safety Open Data	Public accident, vehicle, and casualty datasets.	Primary public dataset
Kaggle traffic accident datasets	Alternative accident datasets for a selected country or city.	Optional source
Open-Meteo API	Historical or current weather variables for enrichment.	Weather source
OpenStreetMap	Road-type and location context where feasible.	Optional geospatial enrichment

Dataset rule: Use only free, legally accessible, public, institutional, or instructor-approved datasets. Include the source name and access instructions in the report.

3. Data Ingestion Tool Information

Use Python/Pandas to ingest large CSV files and Python requests for weather APIs. Apache Airflow should manage staged extraction, joins, and scheduled refreshes. PostgreSQL/PostGIS is recommended for storage.

Minimum ingestion expectations

- Maintain a raw copy of the extracted data.
- Record extraction date, source, file/API status, and row count.
- Handle extraction errors and log failed records.
- Support at least one repeatable or scheduled ingestion workflow.
- Do not manually edit the final analytical dataset.

4. Suggested Data Architecture Design

Accident files + weather API + optional map data -> raw landing zone -> Airflow ETL -> staging and geospatial transformation -> PostgreSQL/PostGIS -> accident mart -> Tableau Public or Streamlit.

Students should prepare their own architecture diagram showing the source layer, ingestion layer, raw/staging layer, transformation layer, storage layer, analytics layer, and later the MLOps layer.

5. ETL and Data Engineering Requirements

- Standardize date, time, severity, road, and location fields.
- Remove duplicates and invalid coordinates.
- Join accident records with weather using time and location.
- Create dimensions for date, road, weather, location, and severity.
- Generate aggregates by hour, day, road type, and weather condition.
- Load analytical and model-ready tables.

6. Suggested Data Visualization

Use **Tableau Public** for business-oriented dashboards and comparative analysis, or **Streamlit** when interactive filtering, user input, or future model prediction is central to the application.

- Severity distribution
- Accidents by hour and weekday
- Weather and road-condition impact
- Location hotspot map
- Vehicle and casualty analysis

7. Assignment Part 1 - Data Pipeline Implementation

Deadline: On or before 7 September 2026, which falls in the first week of September. Students must build a complete data pipeline without adding the final machine-learning deployment at this stage.

Required implementation

- Acquire and document the selected dataset(s).
- Create automated or reproducible ingestion scripts.
- Implement an Apache Airflow DAG, or an approved equivalent ETL workflow.
- Create raw, staging, cleaned, and analytical data layers.
- Apply data-quality checks and produce a rejected-record or error log.
- Load the transformed data into PostgreSQL or another approved open-source database.
- Create at least one analytical table or data mart.
- Develop a Tableau Public or Streamlit dashboard with at least five meaningful views or indicators.
- Prepare an architecture diagram and data dictionary.
- Demonstrate one successful end-to-end pipeline execution.

Part 1 submission package

Item	Required submission

1	Source code and ETL scripts
2	Airflow DAG or approved orchestration workflow
3	Database schema and sample populated tables
4	Dataset source information and access instructions
5	Architecture diagram and pipeline flow
6	Data dictionary and validation rules
7	Tableau Public link/file or Streamlit application
8	Project report of approximately 8-10 pages
9	Screenshots or execution evidence
10	README with setup and run instructions

8. Assignment Part 2 - MLOps Pipeline Extension

After completing Part 1, students will extend the same project into an end-to-end MLOps solution. The Part 2 deadline will be announced separately.

- Predict severity classes using Random Forest, XGBoost, or LightGBM.
- Address class imbalance using weighting or sampling.
- Track recall, macro-F1, and confusion matrices with MLflow.
- Package preprocessing and model as one inference pipeline.
- Deploy with FastAPI and Docker.
- Monitor class drift, location drift, and false-negative rates.

Common Part 2 requirements

- Create reproducible feature-engineering and model-training pipelines.

- Use a chronological or problem-appropriate train/validation/test split.
- Track experiments, parameters, metrics, and artifacts using MLflow.
- Register and version the selected model.
- Serve predictions through FastAPI or an approved API framework.
- Containerize the inference service using Docker.
- Integrate model output into the Streamlit or Tableau-supported workflow where appropriate. •

Monitor input-data quality, drift, model performance, latency, and failures.

- Define retraining criteria and document the model lifecycle.

9. Suggested Mark Distribution

Component	Part 1	Part 2
Problem understanding and design	5	5
Data source and ingestion	10	5
ETL, validation, and storage	20	5

Visualization and interpretation	10	5
Documentation and demonstration	5	5
Model development and evaluation	-	10
MLflow, registry, deployment, and Docker	-	10
Monitoring and retraining design	-	5
Total	**50**	**50**

10. General Instructions

- The project must be completed individually.
- All source code must be written, organized, and explained by the student.
- Public code may be consulted, but copied work without attribution will not be accepted.
- Use environment variables for database credentials and API keys.
- Do not upload private, personally identifiable, or confidential data.
- The final demonstration must show the pipeline from data ingestion to dashboard output.

Late submissions will be handled according to course policy.